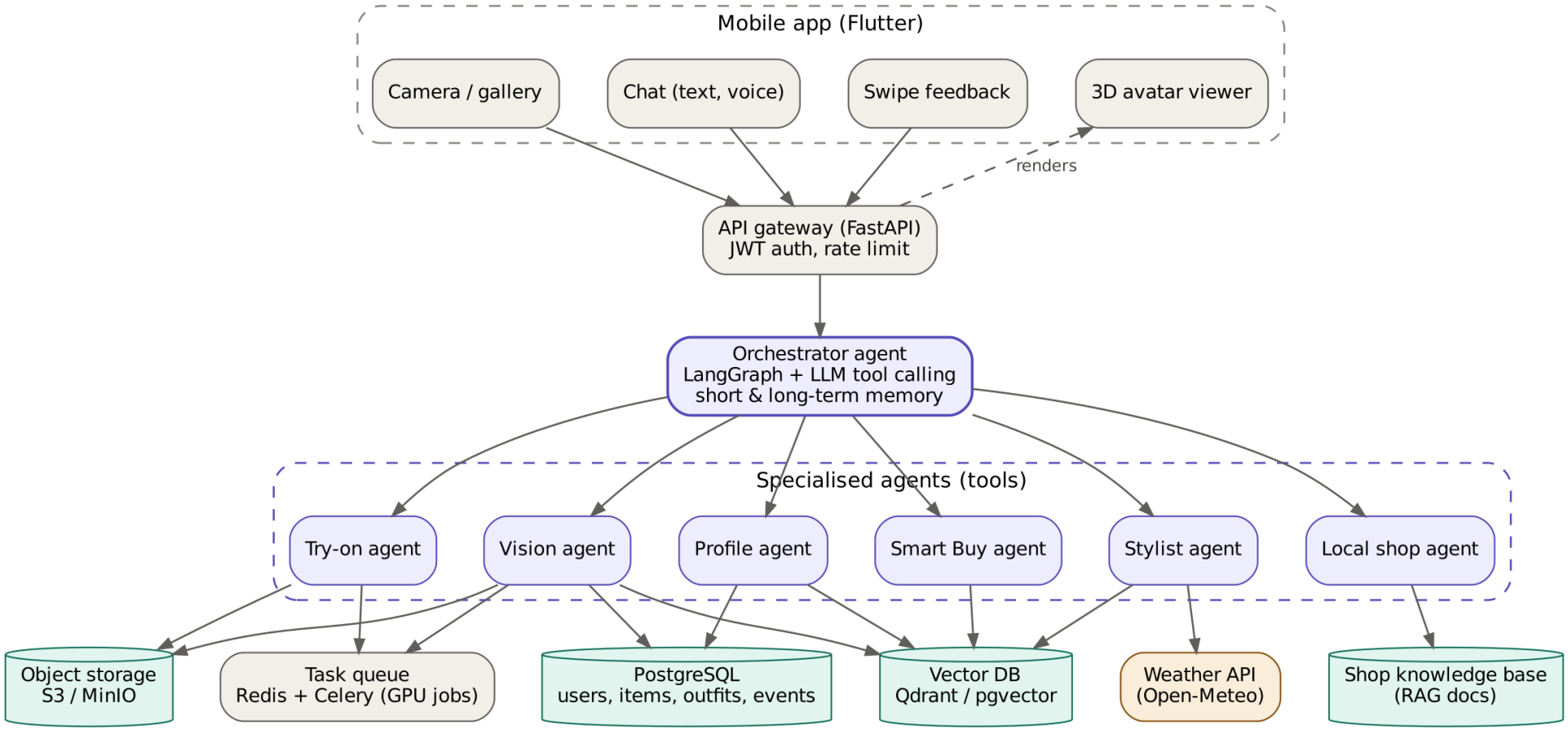
DressMe — Multi-agent pipeline, data & tech stack

Mobile app · agentic architecture · working document for the team of 6 (v1, 10 Oct 2026). Every request from the app reaches one **orchestrator agent**, which calls six specialised agents as tools. Agents share one data contract (garment, outfit, event) and four stores. Heavy GPU work (segmentation, try-on) runs asynchronously through a task queue.



1. Shared data contract (fix in week 1)

All agents read/write these three objects. Keep them versioned in one `schemas/` folder (Pydantic models on the backend, generated Dart classes in Flutter).

Garment

```
{
  "item_id": "itm_8f21",
  "user_id": "usr_042",
  "image_raw": "s3://dressme/raw/itm_8f21.jpg",
  "image_cutout": "s3://dressme/cut/itm_8f21.png",
  "category": "top",
  "subcategory": "shirt",
  "colors": [{"name":"navy","hex":"#1F2A44","share":0.82}],
  "pattern": "solid",
  "material": "linen",
  "season": ["spring","summer"],
  "formality": 0.7,
  "local_tag": null,
  "embedding_id": "vec_8f21",
  "confidence": 0.91,
  "wear_count": 4,
  "last_worn": "2026-10-02",
  "created_at": "2026-09-14T09:12:00Z"
}
```

Outfit

```
{
  "outfit_id": "out_77c",
  "user_id": "usr_042",
  "items": ["itm_8f21","itm_1a90",
            "itm_44de","itm_0b13"],
  "context": {
    "occasion": "interview",
    "date": "2026-10-11",
    "weather": {"temp_c":24,
                "condition":"sunny"}
  },
  "scores": {"compatibility":0.86,
             "style_match":0.79,
             "weather_fit":0.95},
  "explanation": "Navy linen shirt and
   beige chinos read formal but stay
   cool at 24°C.",
  "source": "stylist_agent",
  "tryon_render": "s3://dressme/vto/out_77c.glb"
}
```

Event

```
{
  "event_id": "evt_5521",
  "user_id": "usr_042",
  "type": "outfit_feedback",
  "target_id": "out_77c",
  "value": "accept",
  "channel": "swipe",
  "ts": "2026-10-10T07:41:00Z"
}

type ∈ {outfit_feedback,
        worn, item_added,
        item_deleted,
        tryon_reaction,
        builder_edit,
        smartbuy_check,
        chat_message}
```

2. Orchestrator agent (member 6)

Role. Understands the request, keeps conversation memory, decides which agents to call and in what order, merges results into one answer.

Example input

```
{ "user_id": "usr_042",
  "message": "What do I wear for an
   interview tomorrow?",
  "locale": "en", "city": "Tunis",
  "attachments": [] }
```

Example output

```
{ "intent": "outfit_request",
  "plan": ["get_weather",
           "get_profile",
           "recommend_outfits",
           "render_tryon"],
  "reply": "Here are 3 looks...",
  "cards": ["out_77c","out_78a",
            "out_79f"] }
```

Techs that may be used

Layer	Options
Agent framework	LangGraph (recommended: graph + state + checkpoints), CrewAI, OpenAI Agents SDK
LLM	Claude / GPT-4o-class via API for tool calling; open option: Llama 3.x / Qwen via Ollama or vLLM
Memory	Short-term: LangGraph state; long-term: Postgres + vector store
Guardrails	Pydantic tool schemas, refusal for off-topic (open-domain chatbot rejected in BO8)
Voice	Whisper (speech-to-text) for voice messages

3. Vision agent (member 1)

Role. Digitises garments: background removal, attribute classification, embedding. Also handles 'Outfit From a Photo' and Smart Buy photos.

Example input

```
{ "image": "s3://dressme/raw/
     itm_8f21.jpg",
  "mode": "add_to_wardrobe" }

mode ∈ {add_to_wardrobe,
        smartbuy_probe,
        inspiration_photo}
```

Example output

```
{ "category": "top",
  "subcategory": "shirt",
  "colors": [{"name":"navy",
              "share":0.82}],
  "pattern": "solid",
  "season": ["spring","summer"],
  "confidence": 0.91,
  "embedding": [0.012, -0.33,
                ... 512 floats] }
```

Techs that may be used

Step	Techs
Segmentation	rembg (U²-Net), SAM 2, or a YOLOv8-seg fine-tuned on clothes
Classification	Fine-tuned EfficientNet / ViT; zero-shot with FashionCLIP as baseline
Colour	K-means on cutout pixels (LAB space) + colour-name mapping
Embedding	FashionCLIP / Marqo-FashionSigLIP (512-d)
Datasets	DeepFashion2, Fashionpedia, iMaterialist + own Tunisian set (jebba, kachabia, modest wear)

4. Stylist agent (member 2)

Role. Builds outfits from the real wardrobe for a context (occasion, weather, style). Powers outfit of the day, Remix, 'complete with AI' in the builder.

Example input

```
{ "user_id": "usr_042",
  "occasion": "interview",
  "weather": {"temp_c":24},
  "style_vector": "vec_u042",
  "locked_items": ["itm_8f21"],
  "k": 3 }
```

Example output

```
{ "outfits": [
   {"items":["itm_8f21","itm_1a90",
             "itm_44de","itm_0b13"],
    "compatibility":0.86,
    "why":["formal","heat-friendly",
           "navy+beige harmony"]},
   ...] }
```

Techs that may be used

Step	Techs
Candidate filter	Rules: season, weather, formality range, category slots (top/bottom/shoes/outer)
Compatibility	Siamese / transformer outfit model trained on Polyvore Outfits; baseline: cosine on CLIP embeddings
Diversity	MMR re-ranking so the 3 proposals differ
Explanation	LLM fed with the scores and attributes (no free invention)
Context	Open-Meteo weather API, occasion taxonomy incl. Tunisian occasions (wedding, Aïd, Ramadan evening)

5. Profile & analytics agent (member 3)

Role. Learns the user's style from events, computes wardrobe analytics (wear rate, duplicates, gaps), feeds explanations.

Example input

```
{ "user_id": "usr_042",
  "events": [
   {"type":"outfit_feedback",
    "target_id":"out_77c",
    "value":"accept"},
   {"type":"worn",
    "target_id":"itm_1a90"}] }
```

Example output

```
{ "style_vector": [...512],
  "traits": {"formality":0.62,
             "palette":["navy","beige"],
             "avoid":["neon"]},
  "analytics": {
    "wear_rate": 0.41,
    "never_worn": ["itm_09aa"],
    "duplicates": [["itm_3","itm_7"]],
    "gaps": ["light jacket"] } }
```

Techs that may be used

Step	Techs
Style vector	Exponential moving average of accepted outfit embeddings minus rejected
Learning	Contextual bandit (LinUCB / Thompson) on top of Stylist scores; RL later
Cold start	Swipe onboarding on 15 generated looks
Analytics	SQL + pandas; duplicates = embedding similarity > 0.85
Scheduling	Celery beat nightly jobs

6. Smart Buy agent (member 4)

Role. Decides if a candidate piece complements or duplicates the wardrobe, and how many new outfits it unlocks.

Example input

```
{ "user_id": "usr_042",
  "probe_image": "s3://dressme/
     probe/p_311.jpg",
  "price_tnd": 89 }
```

Example output

```
{ "score": 78,
  "verdict": "good_buy",
  "duplicates": [],
  "new_outfits": 6,
  "pairs_with": ["itm_1a90",
                 "itm_44de"],
  "reasons": ["fills 'light jacket'
              gap", "matches palette"] }
```

Techs that may be used

Step	Techs
Similarity	Qdrant / pgvector k-NN on FashionCLIP embeddings
New outfits	Calls Stylist agent with the probe item locked
Score	Weighted: gap fill, compatibility, redundancy penalty, cost per wear (later)
Sharing	Shareable verdict card (BO7 poll)

7. Local shop agent (member 4)

Role. Answers 'where in Tunis can I find…?' from a curated base of Tunisian boutiques and brands (no live scraping).

Example input

```
{ "query": "white linen shirt
     under 80 TND in Tunis",
  "gap": "white shirt",
  "style_vector": "vec_u042" }
```

Example output

```
{ "results": [
   {"shop":"<partner boutique>",
    "area":"La Marsa",
    "item":"Linen shirt, white",
    "price_tnd":75,
    "match":0.81}] }
```

Techs that may be used

Step	Techs
Knowledge base	Curated CSV/JSON of shops and pieces, entered manually or from partner files
Retrieval	RAG: multilingual embeddings (e5 / bge-m3, handles FR/AR/EN) + hybrid BM25
Filters	Price (TND), city, category, style match
Framework	LlamaIndex or LangChain retriever as a LangGraph tool

8. Try-on agent (member 5)

Role. Shows outfits on a faceless 3D avatar built from the user's measurements. 2D diffusion try-on is the fallback.

Example input

```
{ "user_id": "usr_042",
  "body": {"height_cm":178,
           "chest_cm":98,
           "waist_cm":84,
           "hips_cm":96},
  "items": ["itm_8f21","itm_1a90"] }
```

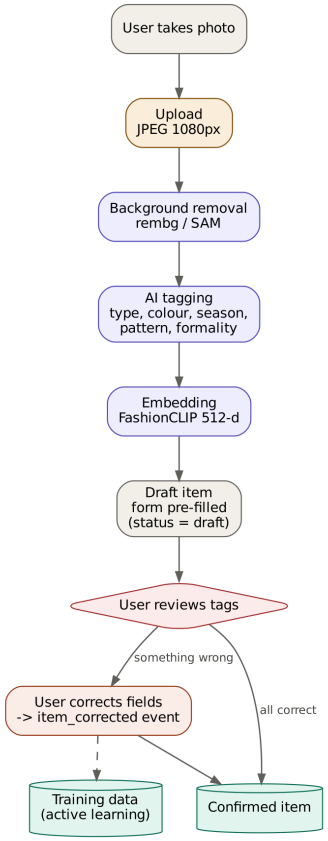
Example output

```
{ "avatar": "s3://dressme/
     avatar/usr_042.glb",
  "render": "s3://dressme/vto/
     out_77c.glb",
  "preview_png": ".../out_77c.png",
  "latency_ms": 4200 }
```

Techs that may be used

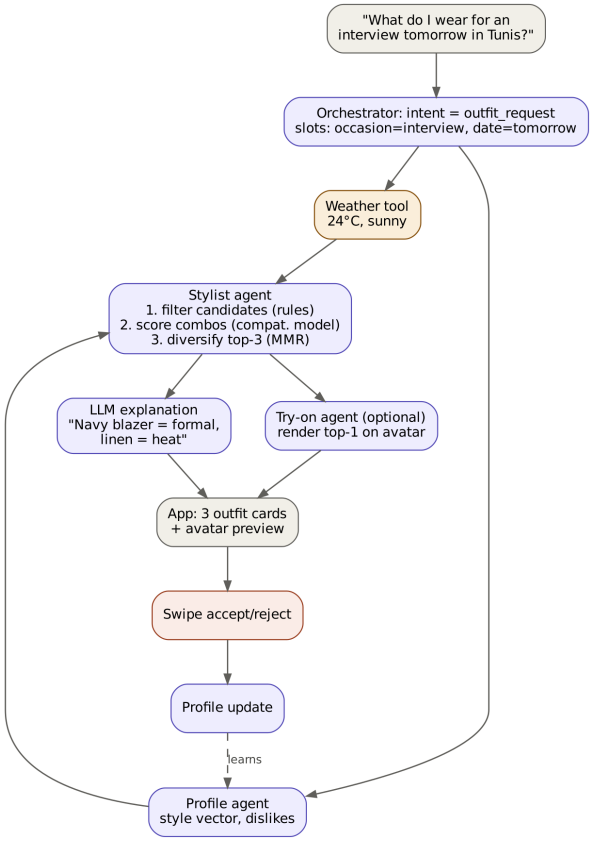
Step	Techs
Body model	SMPL-X shape fit from measurements
Garments	Template meshes per category (shirt, trousers, dress…) + texture from the cutout image
Rendering	glTF/GLB served to Flutter (model_viewer_plus) or Unity/Three.js view
Fallback	2D diffusion VTON: IDM-VTON / OOTDiffusion on GPU
Compute	GPU worker (Colab / cloud GPU) behind Celery queue

9. Flow — adding a garment (AI tags, user corrects)



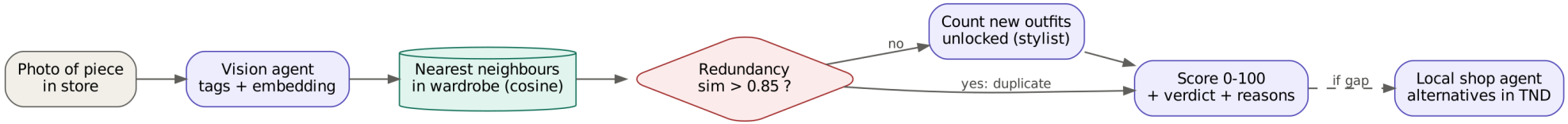
The AI fills every tag; the user always sees a pre-filled draft and only fixes what is wrong. Each correction is logged (item_corrected) and becomes labelled training data. KPI: % of items confirmed with no correction.

10. Flow — outfit request (chat or outfit of the day)



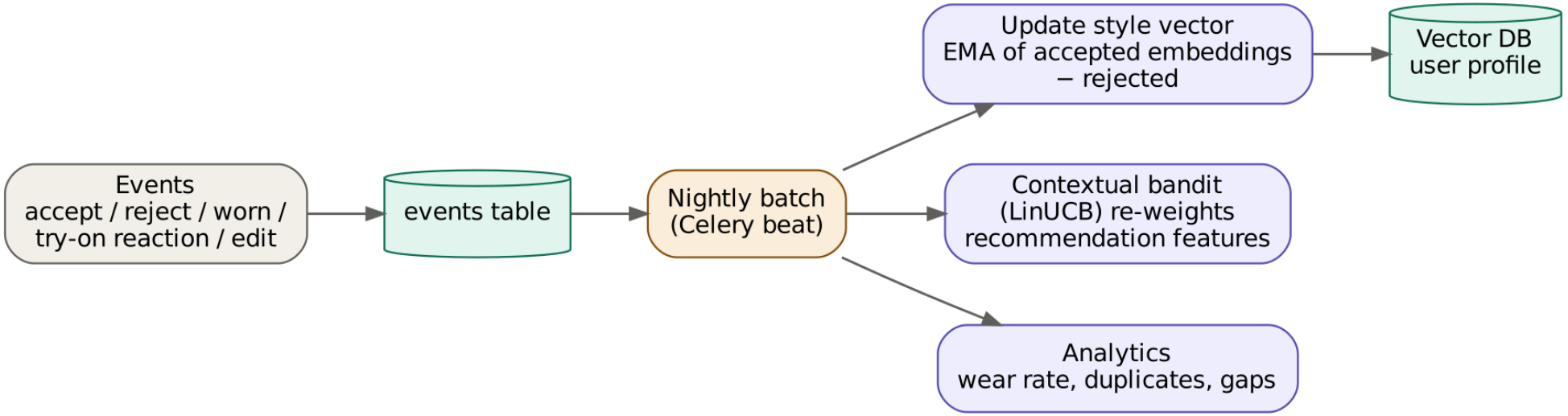
The same path serves the morning outfit (triggered by a scheduled job instead of a message) and chat requests. Every swipe becomes an event for the Profile agent.

11. Flow — Smart Buy check in store



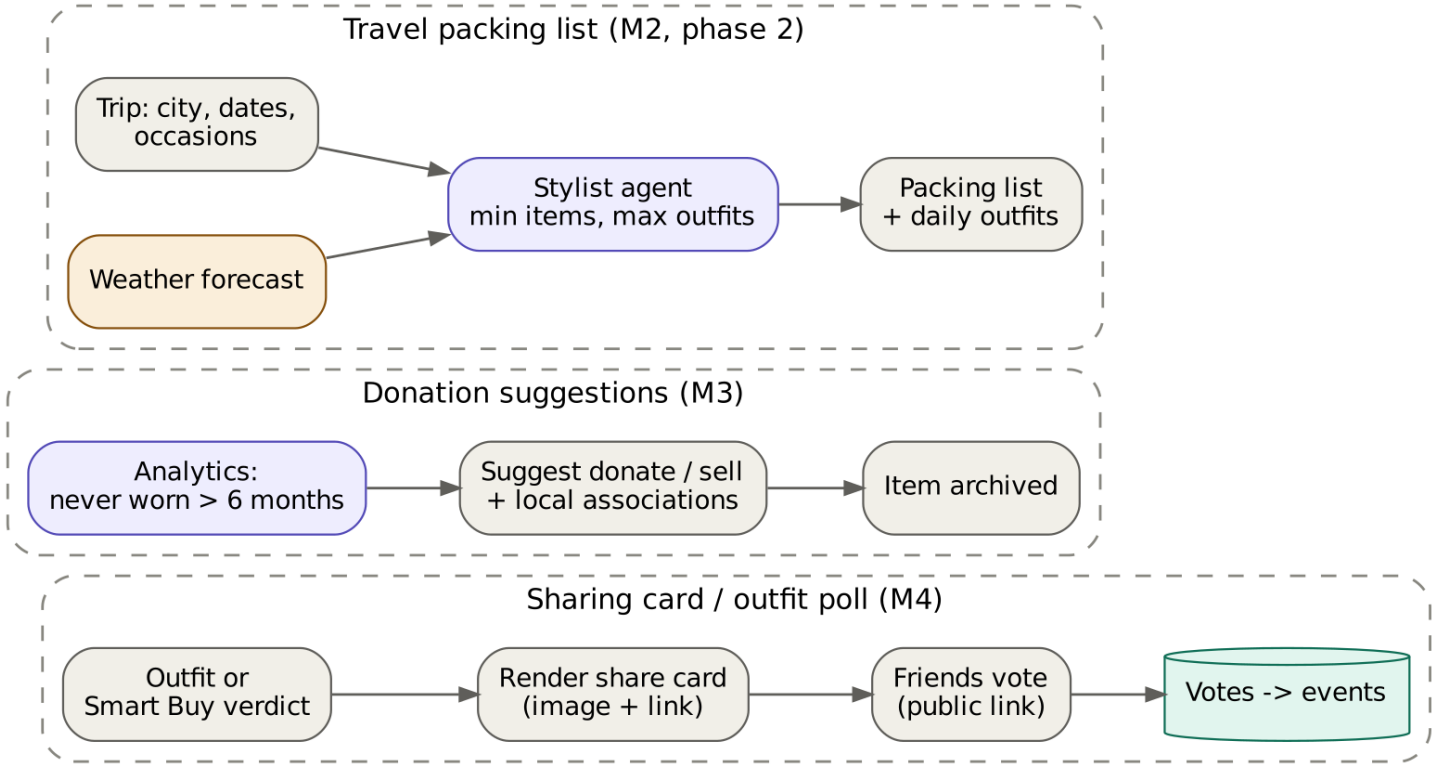
Thresholds (0.85 similarity, 0.6 confidence) are starting values to tune on your validation set.

12. Flow — learning loop



Start with the EMA style vector (simple, explainable); add the bandit once you have enough events.

13. Extra flows — sharing, donation, travel



Low-AI features that reuse existing agents: sharing cards (BO7), donation from wardrobe analytics (BO3), travel packing list as a Stylist mode (phase 2).

14. Global tech stack

Layer	Choice (recommended)	Alternatives
Mobile	Flutter	React Native
Backend API	FastAPI (Python)	Django REST
Agents	LangGraph	CrewAI, AutoGen
LLM	API model with tool calling	Llama / Qwen self-hosted (vLLM, Ollama)
Vision	PyTorch, rembg/SAM 2, FashionCLIP	YOLOv8-seg, timm models
Databases	PostgreSQL + pgvector, MinIO/S3	Qdrant, Firebase Storage
Async jobs	Celery + Redis	RQ, Dramatiq
MLOps	MLflow (experiments), DVC (datasets), Docker	Weights & Biases
Deployment	Docker Compose → cloud VM + GPU worker	Render, Railway, GCP Cloud Run
Monitoring	LangSmith / Langfuse (agent traces), Prometheus	Sentry

15. Ownership and data sets per member

Member	Agent / module	Data they own	Main datasets
1	Vision agent	garments, images	DeepFashion2, Fashionpedia, own Tunisian photos
2	Stylist agent	outfits	Polyvore Outfits, own accepted outfits
3	Profile & analytics	events, style vectors	App events (synthetic first)
4	Smart Buy + Local shop	shop knowledge base	Curated Tunisian shop list
5	Try-on agent	avatars, renders	SMPL-X, VITON-HD (fallback)
6	Orchestrator, API, MLOps	schemas, conversations	Test dialogues per BO

16. Feature map — every selected idea, its flow and its owner

Each member owns at least one AI component and one non-AI feature (UI / CRUD / data).

#	Feature	Type	Agent / flow	Feasibility	Owner	Phase
1	Photograph & save items	AI	Vision agent · flow 9	Easy	M1	MVP
2	AI fills tags, user corrects	AI + UI	Vision · flow 9 (draft review)	Easy	M1	MVP
3	Wardrobe CRUD (edit, delete, mark worn)	CRUD	API wardrobe	Easy	M1	MVP
4	Outfit of the day / on-demand generation	AI	Stylist · flow 10	Medium	M2	MVP
5	Outfit Remix	AI	Stylist (MMR diversity)	Medium	M2	MVP
6	Build your own outfit + save (CRUD)	CRUD + UI	Builder · API outfits	Easy	M2	MVP
7	Complete my outfit with AI	AI	Stylist (locked_items)	Easy	M2	MVP
8	Explainable recommendations	AI	Stylist explain + Profile traits	Easy	M2 / M3	MVP
9	Adaptive style profile	AI	Profile · flow 12	Medium	M3	MVP
10	Wardrobe analytics	Data	Profile agent	Easy	M3	MVP
11	Smart Buy	AI	Smart Buy · flow 11	Medium	M4	MVP
12	Local Tunisian shops	AI (RAG)	Local shop agent	Medium (data by hand)	M4	Phase 2
13	Faceless 3D avatar try-on	AI / 3D	Try-on agent (2D fallback)	Hard — high risk	M5	Phase 2
14	Conversational assistant	AI (LLM)	Orchestrator	Medium	M6	MVP
15	Outfit from a photo	AI	Vision (inspiration) → Stylist	Medium	M1 + M2	Phase 2
16	Tunisian occasions	Data	Stylist context / tags	Easy	M2	MVP
17	Sharing cards / outfit poll	UI	Flow 13	Easy	M4	Phase 2
18	Donation suggestions	Data	Profile analytics · flow 13	Easy	M3	Phase 2
19	Travel packing list	AI	Stylist mode · flow 13	Medium	M2	Phase 2
20	Size & fit advice	—	Backup only (no brand size data)	Not feasible now	—	Backup

17. Task distribution — 6 members

Rule: every member owns **one AI / agent component** (main) and **one non-AI feature** (UI, API/CRUD, data or infra), end to end: model → API route → screen. Each works only in their own folders (see CODEOWNERS).

Member	AI / agent work (main)	Non-AI work (secondary)	Screens (mobile/lib/features)	Repo folders
M1	**Vision agent**: segmentation, garment classification incl. Tunisian categories, colour extraction, FashionCLIP embeddings, outfit-from-photo mode	Wardrobe CRUD API (add, edit, delete, mark worn), AI-tag review flow, image storage (MinIO)	wardrobe/ (camera, grid, tag review)	agents/vision, api/routes/wardrobe.py
M2	**Stylist agent**: candidate filtering, compatibility model, MMR diversity (Remix), complete-with-AI, explanations, Tunisian occasions	Outfits CRUD API (saved & user-built outfits), weather API integration, travel list (phase 2)	outfit_of_day/, builder/	agents/stylist, api/routes/outfits.py
M3	**Profile agent**: style vector (EMA), contextual bandit, cold start, wardrobe analytics (duplicates, gaps, never worn)	Event tracking (every user action), onboarding swipe, donation suggestions (phase 2)	onboarding/, profile/	agents/profile
M4	**Smart Buy agent** (similarity, redundancy, score) + **Local shop agent** (RAG over Tunisian shops)	Build & clean the Tunisian shop dataset, sharing cards / outfit poll	smart_buy/	agents/smartbuy, agents/localshop, api/routes/smartbuy.py
M5	**Try-on agent**: SMPL-X faceless avatar from measurements, garment texturing, GLB export, 2D diffusion fallback	GPU worker setup (Celery queue), 3D viewer integration in Flutter	try_on/	agents/tryon, api/routes/tryon.py
M6 (Aziz)	**Orchestrator agent**: LangGraph, LLM tool calling, memory, guardrails, agent evaluation	API gateway & auth, Docker/infra, CI/CD, shared schemas review, repo management, app shell (theme, navigation)	chat/, app/, core/	orchestrator, api/main.py, shared, infra, .github

Shared responsibilities

Topic	How it is split
Data collection	Each owner collects/labels data for their agent; everyone contributes photos to the common Tunisian garment dataset.
Tests	Each owner writes tests + JSON fixtures for their agent and routes; CI must pass before merge.
Design	Everyone follows the DressMe visual identity (colours, fonts, components); M6 owns theme.dart and core widgets.
Documentation	Each owner writes their agent README and their section of the report; M6 keeps docs/architecture.md.
Code review	Each PR reviewed by one member; changes to shared/schemas reviewed by M6.

Workload balance

M5 (try-on) carries the heaviest R&D, so its non-AI part stays small and tied to the agent. M6 carries the coordination load. M4 owns two agents but both are lighter (vector search + RAG). Phase 2 items (travel list, donation, sharing, local shops, 3D try-on) start only once the MVP flows 9–11 work end to end.

18. Business distribution — each member owns a value for the user

Every member owns one **business mission**: the agent that delivers it, the business objectives it serves, the personas it targets and the KPI they are accountable for. Technical tasks (page 17) follow from this mission.

Member	Business mission	Agent	Business objectives	Personas	KPI owned	What they deliver
M1	**Wardrobe Digitisation** *"Put my wardrobe in my phone in seconds."*	Digitiser agent (Vision)	Digitise the wardrobe (base of all BOs) · BO3 waste	Sarra, Lina, Kenza	% items confirmed without correction · time to add an item	• Agent that photographs, cuts out and tags every piece • AI fills tags → user corrects (review flow) • Tunisian categories (jebba, kachabia…) • Wardrobe management (add, edit, delete, worn) • Outfit from an inspiration photo (with M2)
M2	**Outfit Recommendation** *"Tell me what to wear, now."*	Stylist agent (Recommender)	BO1 personalised outfits · BO2 time saving · Remix	Amine, Kenza, Sarra	Outfit acceptance rate · time to decision	• Agent that recommends outfits from the real wardrobe • Occasion + weather (incl. Tunisian occasions) • Outfit Remix: new combinations • Build-your-own outfit + "complete with AI" • Explain why each outfit works · travel list (ph. 2)
M3	**Style Intelligence & Sustainability** *"Understand my style, show me what I really use."*	Profile agent (Learner + Analyst)	BO4 style understanding · BO3 reduce clothing waste	Lina, Amel	30-day retention · share of wardrobe worn	• Agent that learns the user's style from every like/dislike • Onboarding swipe (first style profile) • Wardrobe insights: never worn, duplicates, gaps • Donation / resale suggestions (ph. 2) • User behaviour tracking for all KPIs
M4	**Smart Shopping & Local Market** *"Should I buy it? Where in Tunisia?"*	Smart Buy agent + Local Market Search agent	BO5 purchase decision · BO6 Tunisian local market · BO7 sharing	Sarra, Amel, Lina	Click-through to partners · partner brands onboarded	• Agent that judges a piece before purchase (duplicate / good buy) • Agent that searches Tunisian boutiques & brands • Build the local brand catalogue (partner contacts) • Affiliation / partnership model (revenue) • Shareable verdict cards & outfit polls
M5	**Virtual Try-On** *"See it on me before I wear or buy it."*	Try-on agent (3D avatar)	BO9 virtual try-on · supports BO5 purchase confidence	Lina, Karim, Kenza	Try-on usage rate · purchases after try-on	• Agent that builds a faceless avatar from measurements • Show outfits / new pieces on the avatar • 2D fallback for the demo • Privacy by design (no face, no body photo stored) • Stylist / pro use case (Karim persona)
M6 (Aziz)	**Personal Assistant & Product Lead** *"One assistant that does everything for me."*	Orchestrator agent (Conversational)	BO8 intelligent assistant · coordinates all BOs	All personas	Conversations per active user · overall KPIs dashboard	• Agent that understands requests and calls the other agents • Chat experience (FR / AR / EN) • Product roadmap, MVP scope, sprint planning • Integration of all agents into one app • Business model & final pitch coordination

Shared by everyone: user interviews and tests with their personas, data for their own agent, their part of the report and of the final pitch.